Traineeship Application

Sprint Report

<TeamNameUnderConstruction>
<Μάριος Ελληνίδης-4926,

Νικόλαος Κωνσταντινίδης-5155 ,

Δημήτριος Καλαθέρης - 5101>

VERSIONS HISTORY

Date	Version	Description	Author
13/03/2025	1	First steps of forming sprintReport	Marios

1. Introduction

This document provides information concerning the sprint of the project.

1.1. Purpose

1.2. Document Structure

The rest of this document is structured as follows. Section 2 describes out Scrum team and specifies the this Sprint's backlog. Section 3 specifies the main design concepts for this release of the project.

2. Scrum team and Sprint Backlog

2.1. Scrum team

Product Owner	Zarras
Scrum Master	Tsoumpaklas
Development Team	Marios Ellinidis , Nikolaos Kwnstantinidis , Dimitrios Kalatheris

2.2. Sprints

Sprint No	Begin Date	End Date	User stories
1	**25/02/2025**	**26/02/2025**	**US1,US2,US3**
2	**26/02/2025**	**27/02/2025**	**US(4,5,7,13)**
3	**28/02/2025**	**28/02/2025**	**US(7,8,10,11)**
4	**1/03/2025**	**4/03/2025**	**US(16,17,18,9,6)**
5	**4/03/2025**	**5/03/2025**	**US(14, 19,20)**
6	**6/03/2025**	**7/03/2025**	**US(12,15)**
7	**7/03/2025**	**8/03/2025**	**US21**

3. Use Cases

3.1. <Use Case 1>

Use case ID	ID_1 ,Create Account
Actors	User
Pre conditions	The connection to the database has been established
Main flow of events	1. The use case starts when the user selects the register button 2. The system redirects to a new webpage with templates for username and password and also to chose what kind of user is the account(professor, student, company , committee member) 3. The user fills the templates given choses what type of user he is and also presses register 4. The system checks if the user already exists 4.1 if the user already exists it informs the user that the account already exists
Post conditions	The User is saved in the database with his password and username

3.2. <Use Case 2>

Use case ID	ID_2 ,Login
Actors	User
Pre conditions	The user has already registered
Main flow of events	1. The use case starts when the user selects to login 2. The system gives the templates to user to fill his username and his password 3. The user fills his username and his password and presses is login again 4. The system compares the username and the password with the ones in the database 4.1. if the the username exists and the password is correct loads the profile dashboard based on what kind of user he is 4.2. else informs the user that either the username or the password was invalid

3.3. <Use Case 3>

Use case ID	ID_3 ,Logout
Actors	User
Pre conditions	The user has already logged in
Main flow of events	1. The use case starts when the user selects to logout from his profile 2. The system logs out the user from the account and redirects him to the homepage
Post Conditions	The user has no access to his profile

3.4. <Use Case 4>

Use case ID	ID_4 ,Create Student Profile
Actors	Student
Pre conditions	The user has logged in and the user is a Student
Main flow of events	1. The use case starts when the student logs in to his account 2. The system loads his profile and provide an update profile button 3. The student presses the update profile button 4. The System redirects to webpage where it provides templates of student name , university id , preferred location , interests and skills 5. The student fills the templates and presses the save button 6. The system saves the students info in the database and redirects the student to the profile dashboard

3.5. <Use Case 5>

Use case ID	ID_5 , Apply for a traineeship
Actors	Student
Pre conditions	The student has updated his profile with all the info required and he hasn't been assigned a traineeship
Main flow of events	1. The use case starts when the student logs in to his account or he has pressed save after updating his account 2. The system provides him with a Apply for a Traineeship button 3. The student presses the Apply for a Traineeship button 4. The System saves this decision to the database
Post condition	The system informs to the student user that he have applied for a traineeship until a traineeship will be assigned to him

3.6. <Use Case 6>

Use case ID	ID_6 , Fill a logbook
Actors	Student
Pre conditions	The student has been assigned a traineeship
Main flow of events	1. The use case starts when the student logs in to student's account 2. The system provides the student with a Report in Logbook button 3. The student clicks the button provided 4. The System provides templates that need to be filled for a new logbook entry which is the date and a description 5. The student fills the templates and clicks save 6. The system redirects the student to the homepage where all the logbook entries are displayed

3.7. <Use Case 7>

Use case ID	ID_7 , Create a company profile
Actors	Company
Pre conditions	The user has logged in and the user is a company
Main flow of events	1. The use case starts when the company logs in to company's account 2. The system loads company's profile and provide an update profile button 3. The company presses the update profile button 4. The System redirects to webpage where it provides templates of company name , and company location 5. The company fills the templates and presses the save button 6. The system saves the company's info in the database and redirects the company to the profile dashboard

3.8. <Use Case 8>

Use case ID	ID_8 , Have access to company's available traineeship positions
Actors	Company
Pre conditions	The company has updated the profile and announced a traineeship position
Main flow of events	1. The use case starts when selects " View available positions" 2. The system searches and views all positions made from this company and have not been assigned to student

3.9. <Use Case 9>

Use case ID	ID_9 , Show Assigned Traineeships
Actors	Company
Pre conditions	A traineeship position has been assigned to a student
Main flow of events	1. The use case starts when the company clicks " View assigned positions" 2. The system searches in the database all the traineeship positions that were announced by the specific company and they are assigned and views them in a list

3.10. <Use Case 10>

Use case ID	ID_10 , Announce an available traineeship position
Actors	Company
Pre conditions	The company has created a profile
Main flow of events	1. The use case starts when the company selects " Announce new position" 2. The system provides the company with templates to fill in for required info about the traineeship position which are the title , the start date , the end date , the description , the required skills and the topics of interest 3. The company fills the template and selects "Save" 4. The system saves the new Traineeship position in the database with it's own info
Post condition	A new available traineeship position has been created in the application

3.11. <Use Case 11>

Use case ID	ID_11 , Delete an available traineeship position
Actors	Company
Pre conditions	The company has announced a traineeship position
Main flow of events	1. The use case starts when the company selects " View available positions" 2. The system searches and views all positions made from this company and have not been assigned to student and also provides a delete button 3. The company selects the delete button 4. The system removes the traineeship position from the list and deletes it from the database
Post condition	The traineeship position is removed from the app

3.12. <Use Case 12>

Use case ID	ID_12 , Company Evalaution
Actors	Company
Pre conditions	The traineeship position has been assigned to a student
Main flow of events	1. The use case starts when the company selects " View available positions" 2. The system searches and views all positions made from this company and have not been assigned to student and provides an "Evaluate" button to each student 3. The company clicks the Evaluate button to a specific student 4. The system provides the company a webpage where the student's info and student's logbook is displayed and also provides him with selection from 1(bad) to 5(perfect) for Motivation , Effectiveness and Efficiency evaluations 5. The company evaluates and clicks submit 6. The system saves the evaluation in the database and displays a summary of the company's evaluation

3.13. <Use Case 13>

Use case ID	ID_13 , Create a professor profile
Actors	Professor
Pre conditions	The user has logged in and the user is a company
Main flow of events	1. The use case starts when the professor logs in to professor's account 2. The system loads professor's profile and provide an update profile button 3. The professor presses the update profile button 4. The System redirects to webpage where it provides templates of professor's full name and interests 5. The professor fills the templates and clicks the save button 6. The system saves the professor's info in the database and redirects the professor to the profile dashboard

3.14. <Use Case 14>

Use case ID	ID_14 Show Traineeship Positions that I Supervise
Actors	Professor
Pre conditions	The professor has been assigned at least one traineeship position
Main flow of events	1. The use case starts when the professor logs in to professor's account 2. The system loads professor's profile and provide a "View Supervised Traineeships " button 3. The professor clicks the button provided by the system 4. The System redirects to webpage where it displays in a list all the traineeship position that he supervises

3.15. <Use Case 15>

Use case ID	ID_15 , Evaluate a traineeship as a supervisor
Actors	Professor
Pre conditions	The professor is a supervisor to at least one traineeship position
Main flow of events	1. The use case starts when the professor clicks “View Supervised Traineeships” button 2. The system loads a list of traineeship positions that the professor supervises and provides an “Evaluate” button to each traineeship position 3. The professor clicks “Evaluate” to a specific traineeship position 4. The System retrieves the specific traineeship position from the database and displays the info of it, also provides him with selection from 1(bad) to 5(perfect) for Motivation , Effectiveness and Efficiency evaluations for student and Facilities , Guidance evaluations for company 5. The professor evaluates and clicks submit 6. The professor fills the templates and clicks the save button 7. The system saves the evaluation in the database and displays a summary of the professor’s evaluation

3.16. <Use Case 16>

Use case ID	ID_16 , Show Traineeship Position Applications
Actors	Committeee member
Pre conditions	Students have applied for traineeship position
Main flow of events	1. The use case starts when the committee member logs in to a committee's account 2. The system loads committee's profile and provide a "Show Traineeship Application " buttons 3. The Committee member clicks the button provided 4. The System redirects to webpage searches in the database students who have applied for a traineeship and display them in a list

3.17. <Use Case 17>

Use case ID	ID_17 , Select a student and search a traineeship position
Actors	Committeee member
Pre conditions	1. Students have applied for traineeship position 2. Students have filled their skills and their interests during the creation of their profile 3. companies have announced traineeship positions 4. Companies have filled their required skills in their announced traineeship positions along with their topic of interests

Main flow of events	1. The use case starts when the committee member clicks “Show Traineeship Applications 2. The system loads a student list that have applied for a traineeship and provides a select button to each student 3. The Committee member clicks select to a specific student 4. The System search the specific student from the database displays student’s info , loads all available traineeship positions from the database and and also provide search by functionalities(interests, location or both) 5. The Committee member selects to search by either by interests , location or both and clicks the “Find position” button 6. The System searches for available positions 6.1 if the committee member has chosen to search based on interests 6.1.1 For every available position 6.1.1.1 If the student has all the required skills of the position 6.1.1.1.1 If the similarity of interests is greater than a given a threshold , add the position to the viewing list 6.2 If the committee member has chosen to search based on location 6.2.1 For every available position 6.2.1.1 If the student has all the required skills of the position 6.2.1.1.1 If the student’s preferred location is similar with the company’s location that has announced the particular position , add the position to the viewing list
Post Conditions	A list of all available traineeship positions are viewed based on search by selection

3.18 < User Case 18 >

Use case ID	ID_18 , Assign Traineeship Position to a student
Actors	Committeee member
Pre conditions	After the search the system has found at least one matching Traineeship position for the student
Main flow of events	1. The use case starts when the committee member has searched for traineeship position either by location or by interests 2. The system views a list of traineeship position based on the search stategy and provides an “Assign button” 3. The Committee member clicks the Assign button to a specific traineeship position 4. The System attaches the specific traineeship position to the selected student
Post condition	The student’s application for a traineeship will be removed , student’s profile is updated with the traineeship position that has been assigned to him and provide him with a logbook functionality

3.19 < User Case 19 >

Use case ID	ID_19 , Select a traineeship position and assign a supervisor
Actors	Committeee member
Pre conditions	1. Students have been assigned a traineeship position 2. Professors have filled their interests during the creation of their profile

Main flow of events	1. The use case starts when the committee member clicks “Show Traineeships In Progresss 2. The system loads a traineeship position list that have been assigned to students from the database and provides an “Assign Professor” button to each traineeship position 3. The Committee member clicks the button provided to a specific traineeship position 4. The System search the specific traineeship position from the database displays traineeship’s info , loads all professors from the database and and also provide assign by functionalities(interests, professor’s load) 5. The Committee member selects to assign by either by interests , professor’s load and clicks the “Assign Professor” button 6. The System searches for professors 6.1 if the committee member has chosen to search based on interests 6.1.1 For every professor 6.1.1.1 If the similarity of interests is greater than a given a threshold and it has the biggest similarity than any other professor assign him to the Traineeship position 6.1.1.2 If the similirity of any professor is smaller than the threshold assign a random professor 6.2 If the committee member has chosen to search based on professor’s load 6.2.1 For every available professor 6.2.1.1 If the professor has the minimum load 6.2.1.1.1 assign the professor as a supervisor
Post Conditions	A supervisor has been assigned to the traineeship position

3.20 < User Case 20 >

Use case ID	ID_20 , View Traineeship Positions that are in progress
Actors	Committeee member
Pre conditions	Traineeships have been assigned to students
Main flow of events	1. The use case starts when the committee member has clicked the show "Traineeships in progress" button 2. The system retrieves from the database all the traineeship positions that have been assigned to students and have not been completed yet and display them in a list

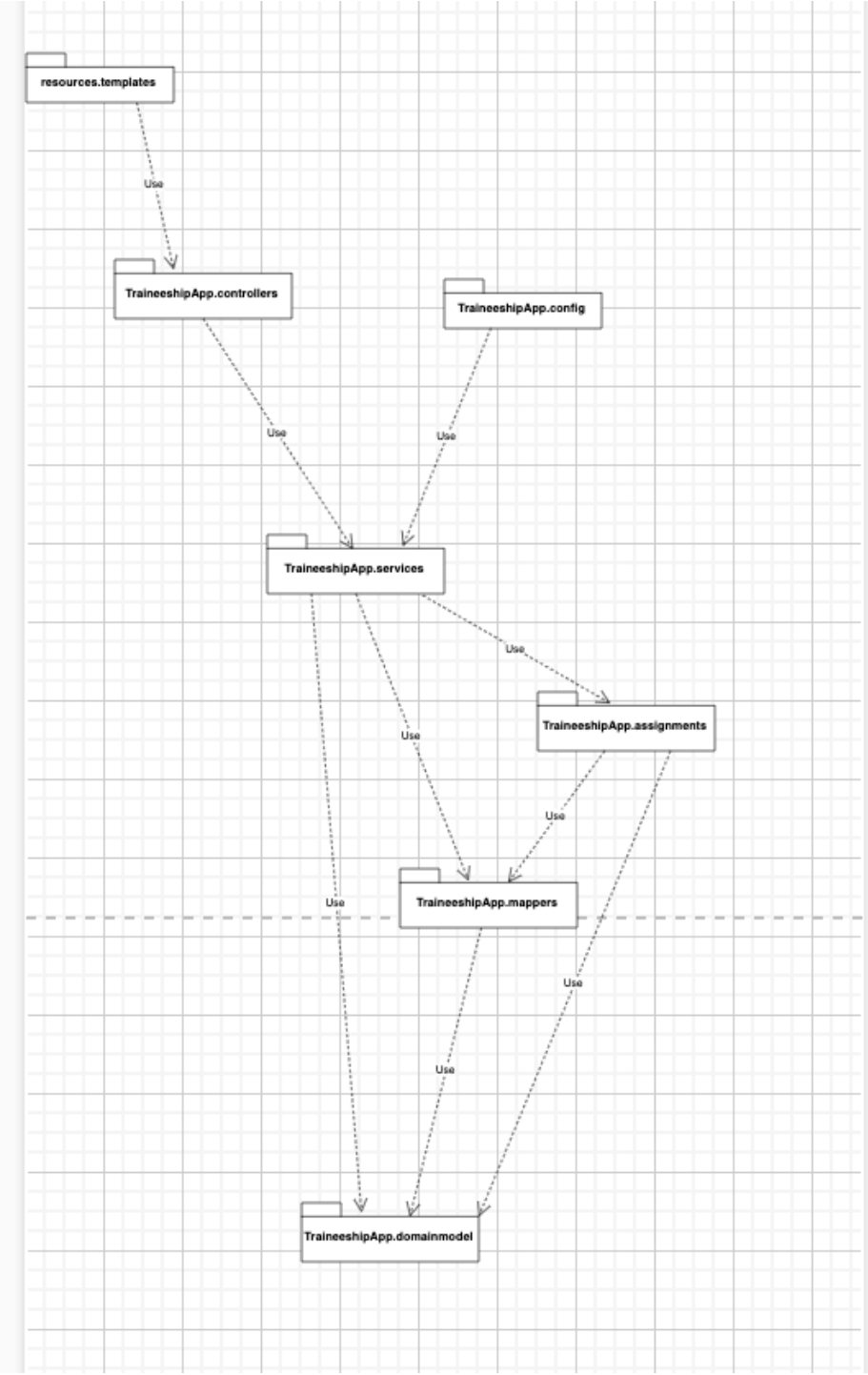
3.21 < User Case 21 >

Use case ID	ID_21 , Complete Traineeship
Actors	Committeee member
Pre conditions	Company and Professor have evaluated the traineeship
Main flow of events	1. The use case starts when the committee member has clicked the show "Show Traineeships in progress" button 2. The system retrieves from the database all the traineeship positions that have been assigned to students and have not been completed yet and display them in a list and also provides a "Complete" button to each one of them 3. The committee member clicks Complete to a specific Traineeship 4. The system redirects to webpage where student's info and logbook is displayed but also a summary of both company's and professor's evaluation for the specific traineeship and also provide pass-fail buttons 5. The committee member clicks pass or fail 6. The system completes the traineeship position and saves the committee's final decision in the database , redirects him to a webpage where it shows a summary of the traineeship and the final result.

4. Design

4.1. Architecture

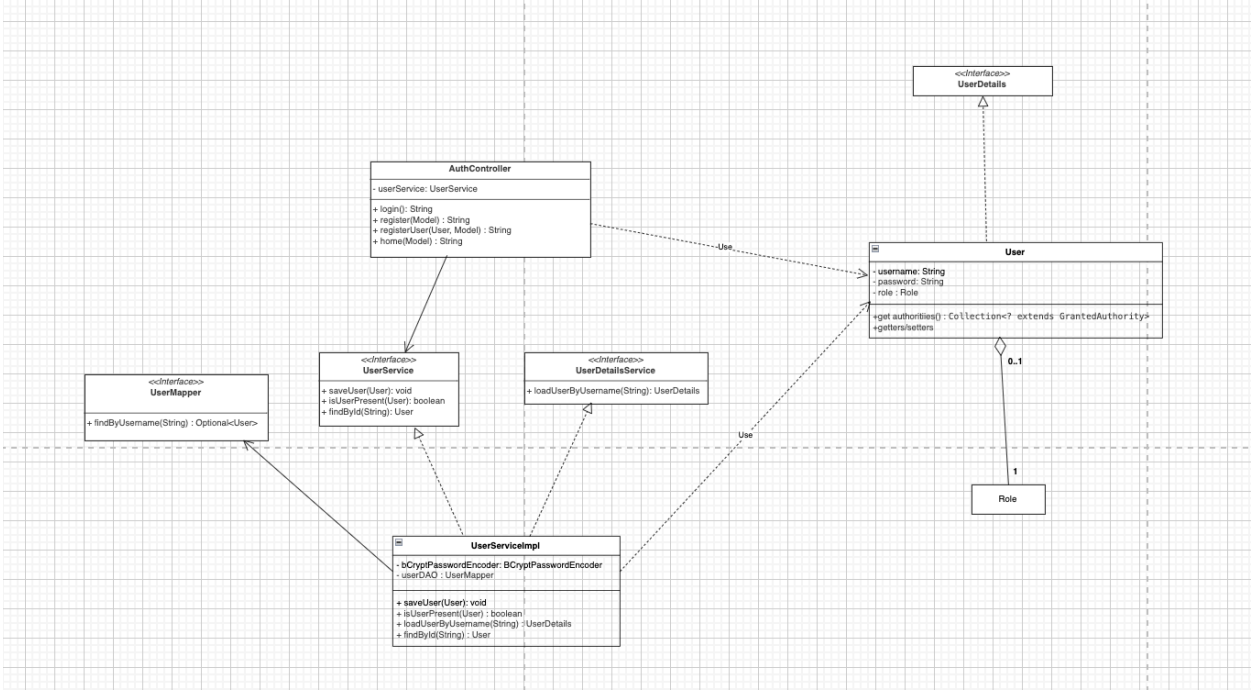
<Specify the overall architecture for this release in terms of a **UML package diagram**.>



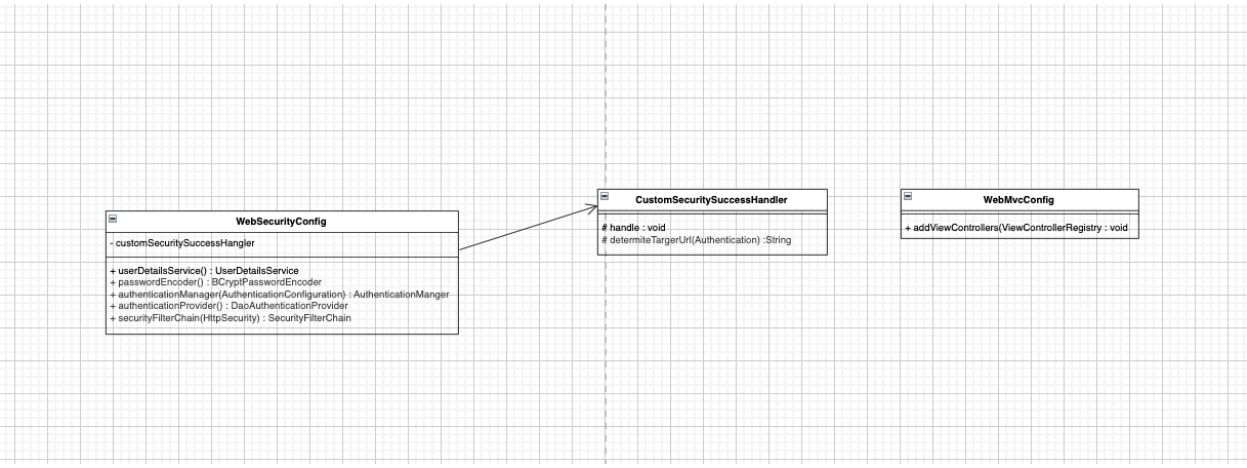
4.2. Design

<Specify the detailed design for this release in terms of **UML class diagrams**.>

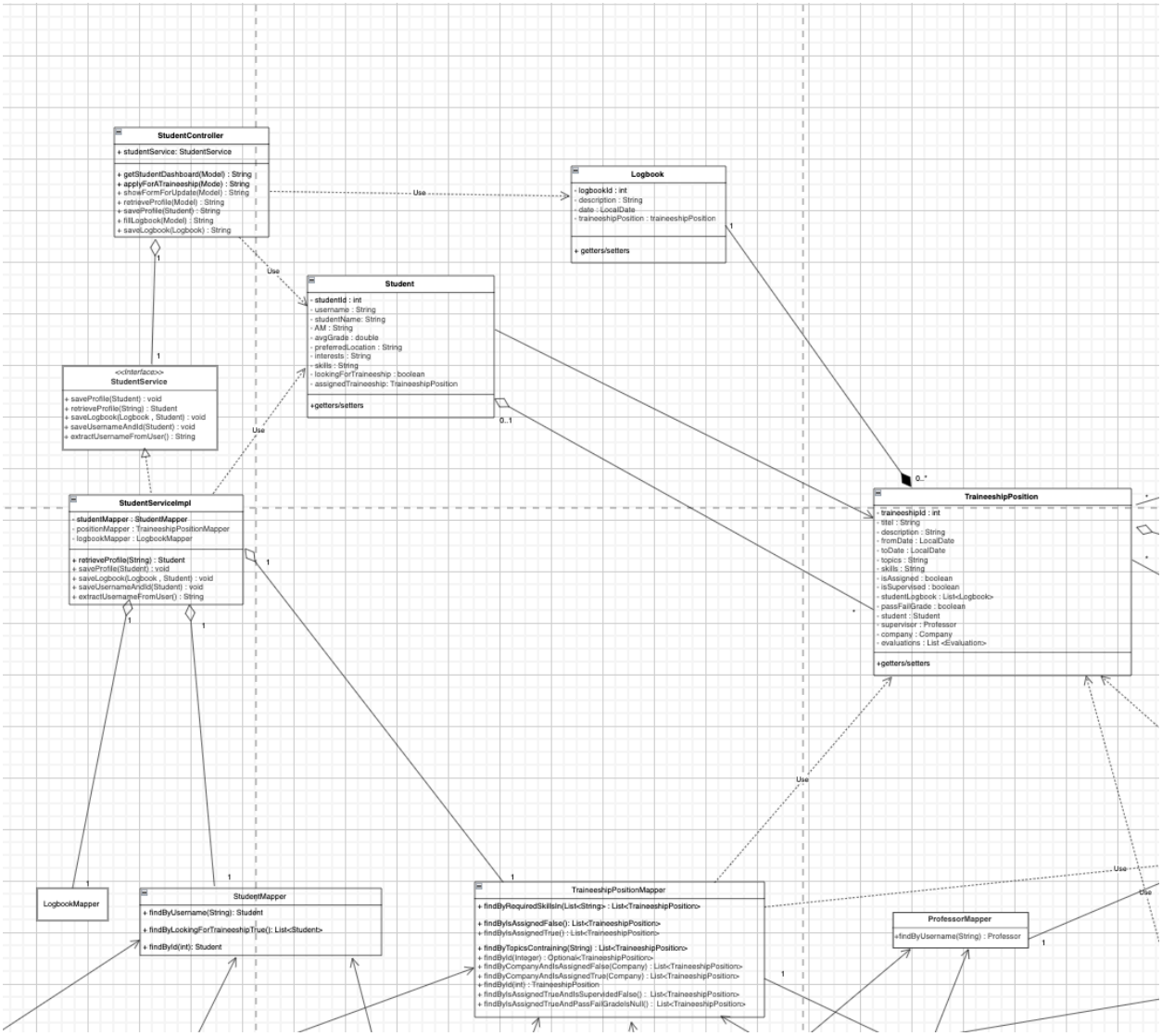
Classes relating User



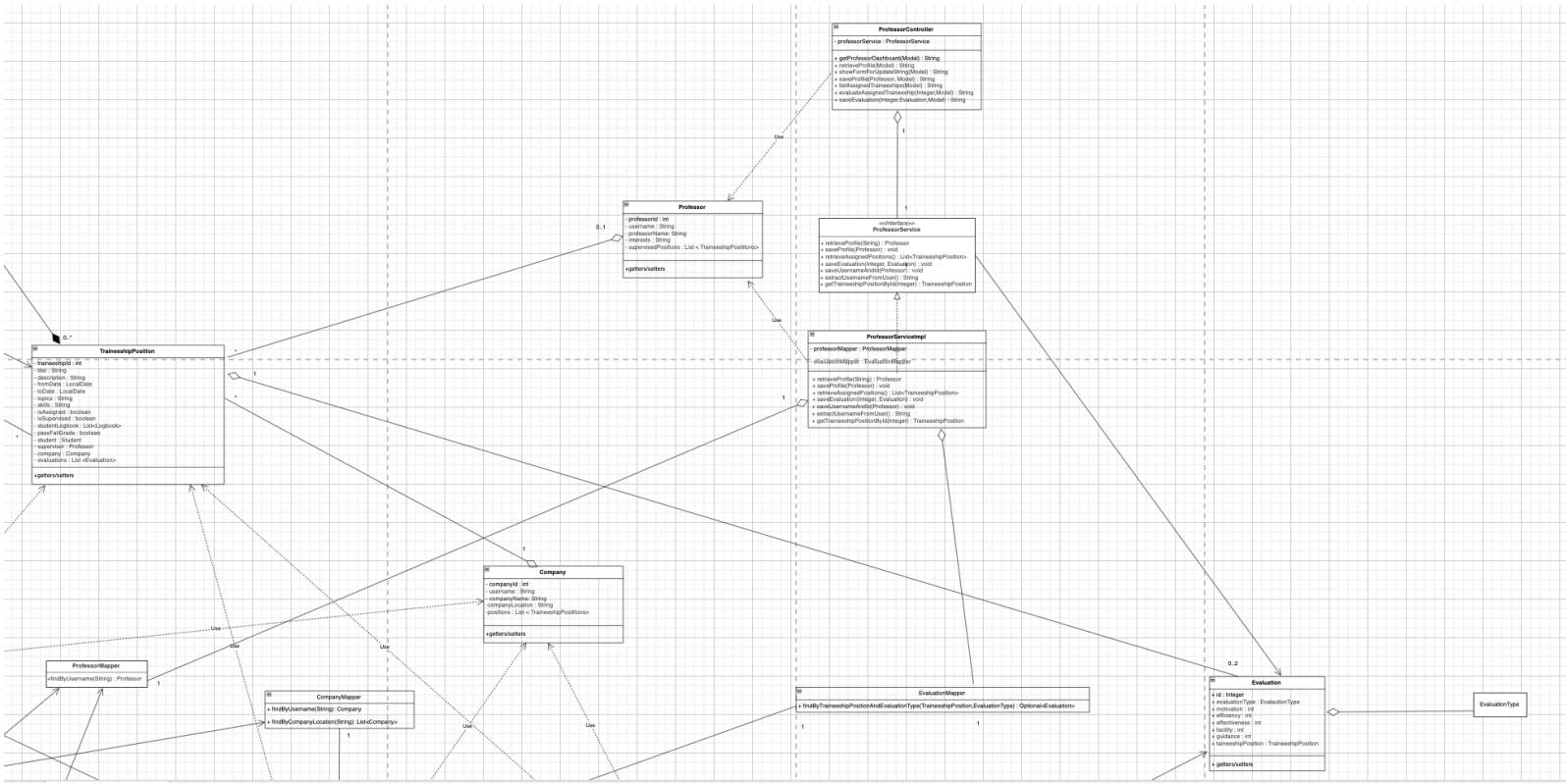
Config implementation



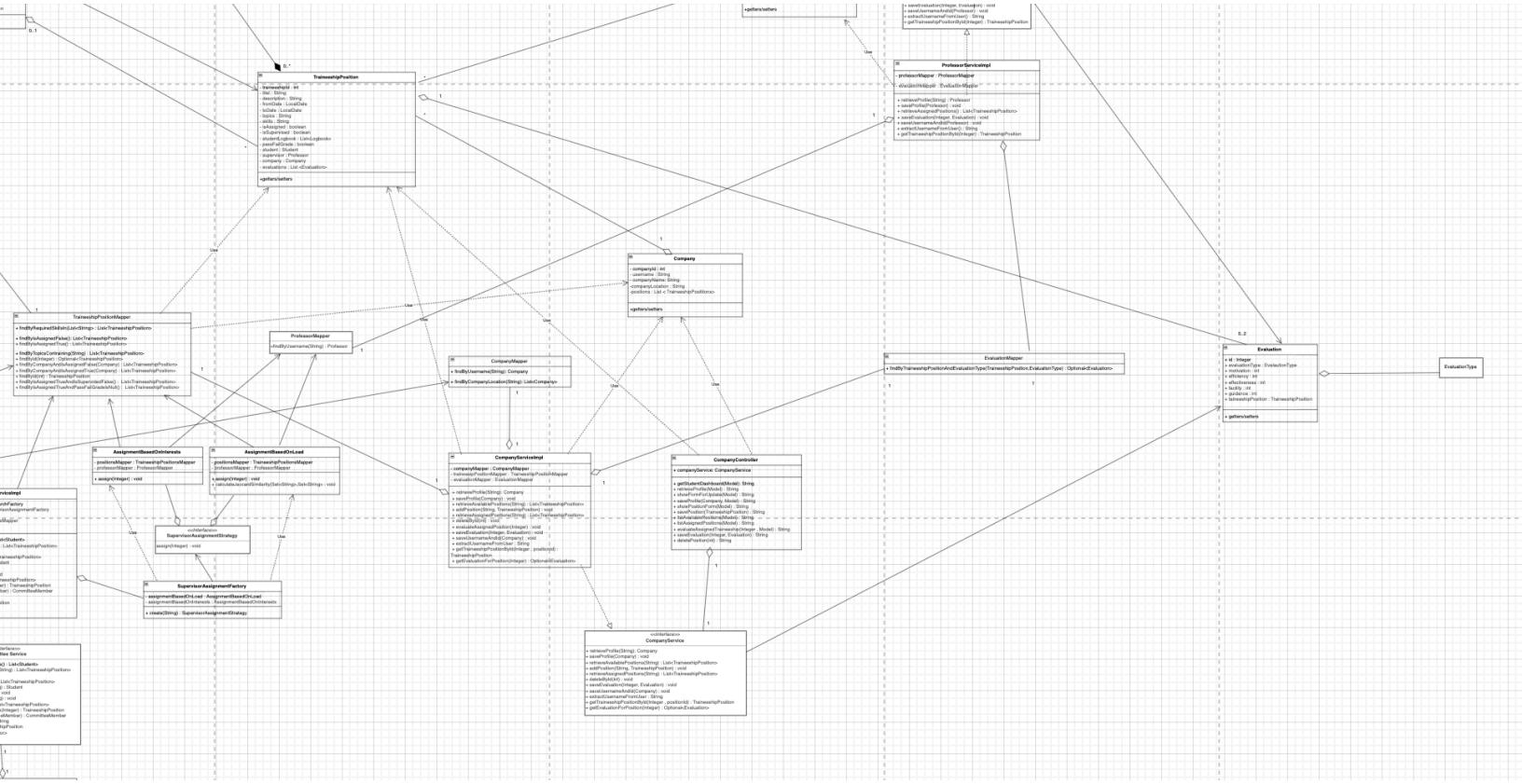
Class diagram regarding Student Entity



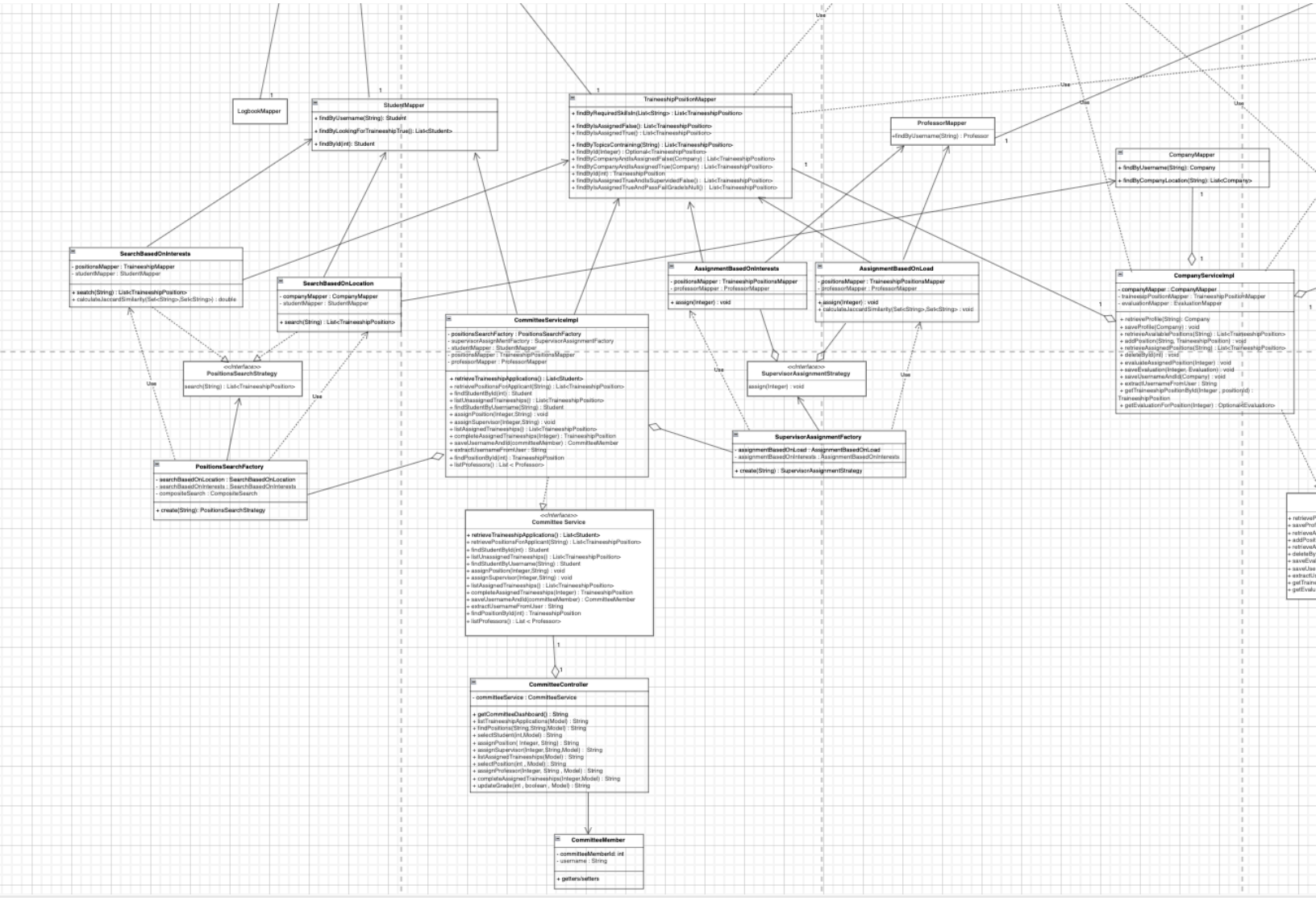
Class diagram regarding Professor Entity



Class diagram regarding Company Entity



Class diagram regarding Committee Member Entity



UML's will be included in the project for more detailed exploration

CRC CARDS

Class Name: WebSecurityConfig	
Responsibilities: ▪ Is responsible for Register and Login functions	**Collaborations:** ▪ CustomSecurityHandler

Class Name: CustomSecurityHandler	
Responsibilities: ▪ Loads the right profile based on what kind of user logged in	**Collaborations:** ▪ CommitteeController ▪ CompanyController ▪ ProfessorController ▪ StudentController

Class Name: AuthController	
Responsibilities: ▪ Ensures the correct loading of the html's based on users actions like Register or LogIn ▪ Ensures the saving of an account	**Collaborations:** ▪ CustomSecurityHandler ▪ WebSecurityConfig ▪ UserService

Responsibilities:	**Collaborations:**
▪ Ensures the correct loading of the html's based on committee's member's actions ▪ Provides necessary object classes from the dababase for each html ▪ Ensures the saving of any object class filled by the committee member through thymeleaf to the database	▪ CommitteeService ▪ CommitteeMember ▪ Student ▪ TraineeshipPosition ▪ Professor ▪ Evaluation ▪ Company ▪ Logbook

Class Name: CompanyController	
Responsibilities:	**Collaborations:**
▪ Ensures the correct loading of the html's based on company's actions ▪ Provides necessary object classes from the dababase for each html ▪ Ensures the saving of any object class filled by the company through thymeleaf to the database	▪ CompanyService ▪ Company ▪ TraineeshipPosition ▪ Evaluation ▪ Student

Class Name: ProfessorController	
Responsibilities:	**Collaborations:**
▪ Ensures the correct loading of the html's based on professor's actions ▪ Provides necessary object classes from the dababase for each html ▪ Ensures the saving of any object class filled by the professor through thymeleaf to the database	▪ ProfessorService ▪ Company ▪ TraineeshipPosition ▪ Evaluation ▪ Logbook ▪ Student ▪ Company

Class Name: StudentController	
Responsibilities: ▪ Ensures the correct loading of the html's based on student's actions ▪ Provides necessary object classes from the dababase for each html ▪ Ensures the saving of any object class filled by the student through thymeleaf to the database	**Collaborations:** ▪ StudentService ▪ Company ▪ Student ▪ Evaluation ▪ Logbook

Class Name: Company	
Responsibilities: ▪ Represent a company in the system ▪ Store and manage company details.	**Collaborations:** ▪ TraineeshipPosition ▪ CompanyContoller ▪ CompanyServiceIml ▪ TraineeshipPositionMapper

Class Name: Evaluation	
Responsibilities: ▪ Store and manage evaluation data for a traineeship position. ▪ Represent different types of evaluations ▪ Maintain scores for various evaluation criteria	**Collaborations:** ▪ TraineeshipPosition ▪ ProfessorService ▪ EvaluationType ▪ CompanyService

Class Name: EvaluationType	
Responsibilities: ▪ Determines if the evaluation was done by the company or by the professor	**Collaborations:** ▪ Evaluation

Class Name: Logbook	
Responsibilities: ▪ It stores a logbook entry of a traineeship	**Collaborations:** ▪ TraineeshipPosition ▪ StudentController

Class Name: Professor	
Responsibilities: ▪ Represent a professor in the system ▪ Store and manage professor details.	**Collaborations:** ▪ ProfessorController ▪ ProfessorServiceImpl ▪ TraineeshipPosition

Class Name: Role	
Responsibilities: ▪ Defines different user roles in the system	**Collaborations:** ▪ User

Class Name: Student	
Responsibilities: ▪ Represent a student in the system ▪ Store and manage student details.	**Collaborations:** ▪ StudentController ▪ StudentServiceImpl ▪ TraineeshipPosition

Class Name: CompanyMapper	
Responsibilities: ▪ Extracts company objects or specific company details from the database ▪ Saves company objects or specific company details in the database ▪ Performs queries relating companies in the database .	**Collaborations:** ▪ SearchBasedOnLocation ▪ CompanyServiceImpl

Class Name: EvaluationMapper	
Responsibilities: ▪ Extracts evalutations from the database ▪ Saves evaluations in the database .	**Collaborations:** ▪ ProfessorServiceImpl ▪ CompanyServiceImpl

Class Name: LogbookMapper	
Responsibilities: ▪ Extracts logbook entries from the database ▪ Saves logbook entries in the database .	**Collaborations:** ▪ StudentServiceImpl

Class Name: ProfessorMapper	
Responsibilities: ▪ Extracts professor objects or specific professor details from the database ▪ Saves professor objects or specific professor details in the database ▪ Performs queries relating professors in the database .	**Collaborations:** ▪ AsssignmentBasedOnInterests ▪ AssignmentBasedOnLoad ▪ ProfessorServiceImpl

Class Name: StudentMapper	
Responsibilities: ▪ Extracts student objects or specific student details from the database ▪ Saves student objects or specific student details in the database ▪ Performs queries relating students in the database .	**Collaborations:** ▪ SearchBasedOnInterests ▪ SearchBasedOnLocation ▪ CommitteeServiceImpl ▪ StudentServiceImpl

Class Name: TraineeshipPositionMapper	
Responsibilities: ▪ Extracts traineeship positions or specific traineeship details from the database ▪ Saves traineeship positions or specific traineeship details in the database ▪ Performs queries relating traineeship positions in the database with various attributes .	**Collaborations:** ▪ StudentServiceImpl ▪ TraineeshipPosition ▪ Company ▪ CompanyServiceImpl ▪ AssignmentBasedOnLoad ▪ AssignmenetBasedOnInterests ▪ CommitteeServiceImpl ▪ SearchBasedOnInterests

Class Name: UserMapper	
Responsibilities: ▪ Extracts user accounts from the database ▪ Save user accounts in the database	**Collaborations:** ▪ UserServiceImpl

Class Name: SupervisorAssignmentStrategy	
Responsibilities: ▪ Determines the assignment strategy of a supervisor to a traineeship	**Collaborations:** ▪ SupervisorAssignmentFactory ▪ AssignmentBasedOnInterests ▪ AssignmentBasedOnLoad

Class Name: SupervisorAssignmentFactory	
Responsibilities: ▪ Ensures the correct implementation of the assign strategy the committee member chose	**Collaborations:** ▪ SupervisorAssignmentFactory ▪ AssignmentBasedOnInterests ▪ AssignmentBasedOnLoad ▪ CommitteeServiceImpl

Class Name: AssignmentBasedOnLoad	
Responsibilities: ▪ Assigns a professor with a minimum load as a supervisor in the traineeship position	**Collaborations:** ▪ SupervisorAssignmentFactory ▪ SupervisorAssignmentStrategy ▪ TraineeshipPosition ▪ ProfessorMapper

Class Name: AssignmentBasedOnInterests	
Responsibilities: ▪ Assigns a professor who has the most common interests with the traineeship position	**Collaborations:** ▪ SupervisorAssignmentFactory ▪ SupervisorAssignmentStrategy ▪ TraineeshipPosition ▪ ProfessorMapper

Class Name: PositionsSearchStrategy	
Responsibilities: ▪ Determines the search strategy for a traineeship position	**Collaborations:** ▪ PositionsSearchFactory ▪ SearchBasedOnInterests ▪ SearchBasedOnLocation

Class Name: PositionsSearchFactory	
Responsibilities: ▪ Ensures the correct implementation of the search strategy the committee member chose	**Collaborations:** ▪ PositionsSearchStrategy ▪ SearchBasedOnInterests ▪ SearchBasedOnLocation ▪ CommitteeServiceImpl

Class Name: SearchBasedOnInterests	
Responsibilities: ▪ Searches for a traineeship position that has plenty of topics that are part of student's interests	**Collaborations:** ▪ PositionsSearchFactory ▪ PositionsSearchStrategy ▪ StudentMapper ▪ TraineeshipPositionMapper

Class Name: SearchBasedOnLocation	
Responsibilities: ▪ Searches for a traineeship positions where the company's location matches the preferred location of the student	**Collaborations:** ▪ PositionsSearchFactory ▪ PositionsSearchStrategy ▪ CompanyMapper ▪ StudentMapper

Class Name: CompositeSearch	
Responsibilities: ▪ Searches for a traineeship positions where the company's location matches the preferred location of the student ▪ Searches for a traineeship position that has plenty of topics that are part of student's interests	**Collaborations:** ▪ PositionsSearchFactory ▪ PositionsSearchStrategy ▪ CompanyMapper ▪ StudentMapper ▪ TraineeshipPositionMapper

Class Name: CommitteeserviceImpl/CommitteeService	
Responsibilities: ▪ Searches for a traineeship positions where the company's location matches the preferred location of the student ▪ Searches for a traineeship position that has plenty of topics that are part of student's interests	**Collaborations:** ▪ PositionsSearchFactory ▪ PositionsSearchStrategy ▪ CompanyMapper ▪ StudentMapper ▪ TraineeshipPositionMapper

Class Name: CommitteeserviceImpl/CommitteeService	
Responsibilities: ▪ Acts as a bridge between the committee controller and the repository ▪ Safely processes and handles data given from the committee member ▪ Stores actions and data from the committee member ▪ Extracts data and informations from the database that are necessary for the using of the app by a committee member	**Collaborations:** ▪ CommitteeController ▪ PositionsSearchStrategy ▪ CompanyMapper ▪ StudentMapper ▪ TraineeshipPositionMapper

Class Name: CompanyServiceImpl/CompanyService	
Responsibilities: ▪ Acts as a bridge between the company controller and the repository ▪ Safely processes and handles data given from the company ▪ Stores actions and data from the company ▪ Extracts data and informations from the database that are necessary for the using of the app by a company	**Collaborations:** ▪ CompanyController ▪ TraineeshipPositionMapper ▪ TraineeshipPosition ▪ CompanyMapper ▪ Company ▪ EvaluationMapper ▪ Evaluation

Class Name: ProfessorServiceImpl/ProfessorService	
Responsibilities: ▪ Acts as a bridge between the professor controller and the repository ▪ Safely processes and handles data given from the professor ▪ Stores actions and data from the professor ▪ Extracts data and informations from the database that are necessary for the using of the app by a professor	**Collaborations:** ▪ Evaluation ▪ EvaluationMapper ▪ TraineeshipPosition ▪ ProfessorController ▪ Professor ▪ ProfessorMapper

Class Name: StudentServiceImpl/StudentService	
Responsibilities: ▪ Acts as a bridge between the student controller and the repository ▪ Safely processes and handles data given from the student ▪ Stores actions and data from the student ▪ Extracts data and informations from the database that are necessary for the using of the app by a student	**Collaborations:** ▪ StudentController ▪ Student ▪ TraineeshipPositionMapper ▪ StudentMapper ▪ LogbookMapper

Class Name: UserServiceImpl/UserService	
Responsibilities: ▪ Acts as a bridge between the user controller and the repository ▪ Safely processes and handles data given from the user during the register and login process ▪ Extracts user nformations from the database for any kind of user	**Collaborations:** ▪ UserMapper ▪ UserService ▪ UserDetailsService ▪ User